

PART 1 — FUNCTIONAL DEPENDENCIES

1. ADMIN

Primary Key : admin_id

Full FD Set :

FD1 : admin_id $\rightarrow$ name

FD2 : admin_id $\rightarrow$ email

Candidate Keys : {admin_id}, {email} (email is UNIQUE)

Minimal (Canonical) FD Set :

admin_id $\rightarrow$ {name, email}

Notes :

- email is a candidate key; email $\rightarrow$ admin_id is also valid.
- No non-trivial FDs between non-key attributes exist.

2. STAFF

Primary Key : staff_id

Full FD Set :

FD1 : staff_id $\rightarrow$ name

FD2 : staff_id $\rightarrow$ contact_no

FD3 : staff_id $\rightarrow$ working_hours

FD4 : staff_id $\rightarrow$ working_days

FD5 : staff_id $\rightarrow$ salary

FD6 : staff_id $\rightarrow$ role

FD7 : staff_id $\rightarrow$ managed_by

Candidate Keys : {staff_id}

Minimal (Canonical) FD Set :

staff_id $\rightarrow$ {name, contact_no, working_hours, working_days, salary, role, managed_by}

Important Note on role $\rightarrow$ salary :

- Same role can have different salaries $\rightarrow$ role does NOT determine salary.
- Therefore: role $\rightarrow$ salary (not a valid FD)
- salary is directly determined by staff_id only.

3. WARDEN (subtype of Staff)

Primary Key : staff_id (also FK $\rightarrow$ Staff)

Full FD Set :

FD1 : staff_id $\rightarrow$ assigned_wing

Minimal FD Set : staff_id $\rightarrow$ assigned_wing

4. CLEANING_STAFF (subtype of Staff)

Primary Key : staff_id (also FK → Staff)

Full FD Set :

FD1 : staff_id → working_area

Minimal FD Set :

staff_id → working_area

5. PLUMBER (subtype of Staff)

Primary Key : staff_id (also FK → Staff)

Full FD Set : None (no non-key attributes)

Minimal (Canonical) FD Set : trivial — only PK exists.

6. CARPENTER (subtype of Staff)

Primary Key : staff_id (also FK → Staff)

Full FD Set : None (no non-key attributes)

Minimal (Canonical) FD Set : trivial — only PK exists.

7. ELECTRICIAN (subtype of Staff)

Primary Key : staff_id (also FK → Staff)

Full FD Set : None (no non-key attributes)

Minimal (Canonical) FD Set : trivial — only PK exists.

8. WING

Primary Key : wing_id

Full FD Set :

FD1 : wing_id → wing_name

FD2 : wing_id → type

FD3 : wing_id → floors

FD4 : wing_name → wing_id (wing_name is unique → candidate key)

FD5 : wing_name → type

FD6 : wing_name → floors

Candidate Keys : {wing_id}, {wing_name}

Minimal FD Set :

wing_id $\rightarrow$ {wing_name, type, floors}

wing_name $\rightarrow$ {wing_id, type, floors}

Notes :

- Both wing_id and wing_name are superkeys.
- No transitive or partial dependencies exist.

9. ROOMS(Weak Entity)

Primary Key : (room_no, Wing_ID) — COMPOSITE KEY

Full FD Set :

FD1 : {room_no, Wing_ID} $\rightarrow$ capacity

FD2 : {room_no, Wing_ID} $\rightarrow$ availability

FD3 : {room_no, Wing_ID} $\rightarrow$ wing_id

Candidate Keys : {room_no, Wing_ID}

Minimal FD Set :

{room_no, Wing_ID} $\rightarrow$ {capacity, availability, wing_id}

10. STUDENT

Primary Key : student_id

Full FD Set :

FD1 : student_id $\rightarrow$ name

FD2 : student_id $\rightarrow$ dob

FD3 : student_id $\rightarrow$ contact

FD4 : student_id $\rightarrow$ email

FD5 : student_id $\rightarrow$ branch

FD6 : student_id $\rightarrow$ emergency_contact

FD7 : student_id $\rightarrow$ father_contact

FD8 : student_id $\rightarrow$ father_name

FD9 : student_id $\rightarrow$ year

FD10 : student_id $\rightarrow$ health_details

FD11 : student_id $\rightarrow$ status

FD12 : student_id $\rightarrow$ program

FD13 : student_id $\rightarrow$ resident_of

FD14 : student_id $\rightarrow$ hmc_flag

FD15 : student_id $\rightarrow$ gender

FD16 : email $\rightarrow$ student_id (candidate key)

FD17 : contact $\rightarrow$ student_id (candidate key)

Candidate Keys : {student_id}, {email}, {contact}

Minimal FD Set :

student_id → {name, dob, contact, email, branch, emergency_contact,
father_contact, father_name, year, health_details,
status, program, resident_of, hmc_flag, gender}

11. ATTENDANCE(Weak)

Primary Key : (student_id, date) — COMPOSITE KEY

Full FD Set :

FD1 : {student_id, date} → status

Partial Dependency Check :

student_id → status ? NO (same student has different status on different dates)

date → status ? NO (same date has different status for different students)

Conclusion : status depends on the FULL composite key only.

Candidate Keys : {student_id, date}

Minimal FD Set :

{student_id, date} → status

12. OVERNIGHT_LEAVE

Primary Key : leave_id

Full FD Set :

FD1 : leave_id → request_date

FD2 : leave_id → from_date

FD3 : leave_id → to_date

FD4 : leave_id → return_timestamp

FD5 : leave_id → status

FD6 : leave_id → student_id

FD7 : leave_id → approved_by

Candidate Keys : {leave_id}

Minimal FD Set :

leave_id → {request_date, from_date, to_date, return_timestamp,
status, student_id, approved_by}

13. COMPLAINT

Primary Key : comp_id

Full FD Set :

- FD1 : comp_id → type
- FD2 : comp_id → status
- FD3 : comp_id → description
- FD4 : comp_id → reporting_date
- FD5 : comp_id → assigned_by
- FD6 : comp_id → student_id
- FD7 : comp_id → solved_by

Candidate Keys : {comp_id}

Minimal FD Set :

- comp_id → {type, status, description, reporting_date,
assigned_by, student_id, solved_by}

14. PARCEL

Primary Key : parcel_id

Full FD Set :

- FD1 : parcel_id → arrival_date
- FD2 : parcel_id → timestamp
- FD3 : parcel_id → status
- FD4 : parcel_id → received_date
- FD5 : parcel_id → student_id
- FD6 : parcel_id → handover_by

Candidate Keys : {parcel_id}

Minimal FD Set :

- parcel_id → {arrival_date, timestamp, status, received_date,
student_id, handover_by}

15. VISITOR

Primary Key : visitor_id

Full FD Set :

- FD1 : visitor_id → name
- FD2 : visitor_id → contact_no
- FD3 : visitor_id → aadhar_no
- FD4 : visitor_id → stay_duration
- FD5 : visitor_id → check_in
- FD6 : visitor_id → check_out

FD7 : visitor_id → request_status
FD8 : visitor_id → approved_by
FD9 : visitor_id → allocated_room
FD10: aadhar_no → visitor_id (Aadhaar is nationally unique → candidate key)

Candidate Keys : {visitor_id}, {aadhar_no}

Minimal FD Set :

visitor_id → {name, contact_no, aadhar_no, stay_duration, check_in,
check_out, request_status, approved_by, allocated_room}
aadhar_no → visitor_id

16. SUGGESTION

Primary Key : suggestion_id

Full FD Set :

FD1 : suggestion_id → description
FD2 : suggestion_id → date
FD3 : suggestion_id → visitor_id

Candidate Keys : {suggestion_id}

Minimal FD Set :

suggestion_id → {description, date, visitor_id}

17. PAYMENT

Primary Key : transaction_id

Full FD Set :

FD1 : transaction_id → method
FD2 : transaction_id → date

Candidate Keys : {transaction_id}

Minimal FD Set :

transaction_id → {method, date}

18. STUDENT_PAYMENT

Primary Key : transaction_id (also FK → Payment)

Full FD Set :

FD1 : transaction_id → student_id
FD2 : transaction_id → amount

Candidate Keys : {transaction_id}

Minimal FD Set :

transaction_id → {student_id, amount}

19. VISITOR_PAYMENT

Primary Key : transaction_id (also FK → Payment)

Full FD Set :

FD1 : transaction_id → visitor_id

FD2 : transaction_id → amount

Candidate Keys : {transaction_id}

Minimal FD Set :

transaction_id → {visitor_id, amount}

PART 2 — NORMALIZATION PROOFS

FIRST NORMAL FORM (1NF)

Definition :

A relation is in 1NF if:

All attributes contain only atomic (indivisible) values.

Proof for each table :

ADMIN

- admin_id, name, email — all single-valued strings.
- IN 1NF

STAFF

- working_hours stores a single shift string (e.g., "6 AM - 2 PM").
 - working_days stores a single string (e.g., "Mon-Sun").
 - No multivalued attributes.
- IN 1NF

WARDEN / CLEANING_STAFF / PLUMBER / CARPENTER / ELECTRICIAN

- Each is a single-row-per-staff subtype with atomic attributes.
- ALL IN 1NF

WING

- wing_name, type, floors — all atomic.
→ IN 1NF

ROOMS

- availability is a single BOOLEAN.
- No repeating groups.
→ IN 1NF

STUDENT

- health_details is stored as a single VARCHAR string per student (e.g., "Asthma"), NOT as a list.
- All other attributes are single-valued.
→ IN 1NF

ATTENDANCE

- status is a single ENUM value per (student, date).
- Composite PK: (student_id, date)
→ IN 1NF

OVERNIGHT_LEAVE

- return_timestamp is a single TIMESTAMP value.
- All other attributes are single-valued.
→ IN 1NF

COMPLAINT

- description is a single TEXT value per complaint.
- All other attributes are single-valued.
→ IN 1NF

PARCEL

- timestamp stores a single TIME per parcel.
- All other attributes are single-valued.
→ IN 1NF

VISITOR

- aadhar_no stored as a single VARCHAR.
- All other attributes are single-valued.
→ IN 1NF

SUGGESTION

- description is a single TEXT per suggestion.
- All other attributes are single-valued.
→ IN 1NF

PAYMENT

- All attributes are atomic.
→ IN 1NF

STUDENT_PAYMENT / VISITOR_PAYMENT

- amount is a single NUMERIC value.
- All other attributes are single-valued.
→ BOTH IN 1NF

SECOND NORMAL FORM (2NF)

Definition :

A relation is in 2NF if:

- It is in 1NF.
- Every non-key attribute is FULLY functionally dependent on the entire primary key (no partial dependencies).

Note : 2NF is only violated when there is a composite primary key.
Single-attribute PK tables are trivially in 2NF.

1. ATTENDANCE

Composite PK : (student_id, date)

Non-key attr : status

Check: student_id → status ?

Student 20240001 on 2026-04-01 → Present

Student 20240001 on 2026-04-03 → Absent

Same student_id, DIFFERENT status on different dates.

Therefore, student_id alone does NOT determine status.

Check: date → status ?

On 2026-04-01: student 20240001 → Present, student 20240002 → Absent

Same date, DIFFERENT status for different students.

Therefore, date alone does NOT determine status.

Check: {student_id, date} → status ?

For a given student on a given date, exactly ONE status exists.

Therefore, status is FULLY dependent on the composite key.

2. ROOMS

Composite PK : (wing_id, room_no)

Non-key attrs: capacity, availability

Check: wing_id $\rightarrow$ capacity ?

Wing 1 has rooms with capacity 2 AND capacity 3.
Same wing_id, DIFFERENT capacities for different rooms.
Therefore, wing_id alone does NOT determine capacity.

Check: wing_id $\rightarrow$ availability ?

Wing 1 has rooms that are TRUE and rooms that are FALSE for availability.
Therefore, wing_id alone does NOT determine availability.

Check: room_no $\rightarrow$ capacity ?

room_no = 101 exists in Wing 1 (capacity 2) AND Wing 2 (capacity 2).
room_no = 102 exists in Wing 1 (capacity 3) AND Wing 2 (capacity 3).
Since room_no is only a PARTIAL KEY, it is NOT unique
across the table — same room_no appears in multiple wings.
Therefore, room_no alone does NOT uniquely determine capacity.

Check: room_no $\rightarrow$ availability ?

Same argument — room_no 101 in Wing 1 may be TRUE while in Wing 2 FALSE.
Therefore, room_no alone does NOT determine availability.

Check: {wing_id, room_no} $\rightarrow$ capacity ?

A specific room within a specific wing has exactly ONE capacity value.

Check: {wing_id, room_no} $\rightarrow$ availability ?

A specific room within a specific wing has exactly ONE availability value.

THIRD NORMAL FORM (3NF)

Definition :

A relation is in 3NF if:

- (a) It is in 2NF.
- (b) There are no transitive dependencies — no non-key attribute determines another non-key attribute.
i.e., for every FD $X \rightarrow Y$, X must be a superkey OR Y must be a prime attribute (part of some candidate key).

STAFF

Potential transitive FD: role $\rightarrow$ salary ?

Data evidence:

role = Warden, shift = morning $\rightarrow$ salary = 40,000

role = Warden, shift = night $\rightarrow$ salary = 42,000

Since the same role maps to TWO different salaries,
role does NOT functionally determine salary.

Therefore, role → salary, NO transitive dependency via role.

STUDENT

Potential transitive FD: resident_of → (room attributes) ?

Room attributes are stored only in the ROOMS, not in STUDENT.

Therefore, No room attributes appear as non-key attributes in STUDENT.

→ NO transitive dependency.

Potential transitive FD: year → program ?

year = 1 can have a program = BTech (multiple branches exist at year 1).

year does NOT uniquely determine the program.

Therefore, year does not uniquely define the program → NO transitive dependency.

VISITOR

Potential transitive FD: allocated_room → (room attributes) ?

Room attributes are stored only in ROOMS, not in VISITOR.

→ NO transitive dependency.

COMPLAINT

Potential transitive FD: type → solved_by ?

type = 'Electrical' can be solved by any of the Electricians.

type does NOT uniquely determine solved_by.

Therefore, type does not define solved_by → NO transitive dependency.

PAYMENT TABLE DESIGN JUSTIFICATION

Without decomposition, a unified Payment table might look like:

Payment(transaction_id, method, date, student_id, visitor_id, amount)

This would introduce:

- NULL student_id for visitor payments
- NULL visitor_id for student payments
- Design ambiguity

Solution: Decompose into:

Payment(transaction_id, method, date, status, receipt_no)

Student_Payment(transaction_id, student_id, amount)

Visitor_Payment(transaction_id, visitor_id, amount)

This eliminates the payer-type ambiguity with no transitive dependencies.

ALL OTHER TABLES

All remaining tables (Wing, Rooms, Warden, Cleaning_Staff, Suggestion, Overnight_Leave, Parcel, Attendance, Admin) have :

- No non-key attribute determines another non-key attribute.
 - All non-key attributes depend DIRECTLY on the primary key.
- NO transitive dependencies.

BOYCE-CODD NORMAL FORM (BCNF)

Definition :

A relation is in BCNF if:

For every non-trivial functional dependency $X \rightarrow Y$,

X must be a SUPERKEY of the relation.

(BCNF is stronger than 3NF — 3NF allows Y to be prime; BCNF does not.)

Checking tables with multiple candidate keys :

WING

Candidate keys : {wing_id}, {wing_name}

FDs:

wing_id $\rightarrow$ {wing_name, type, floors}

Is wing_id a superkey? YES (it is a candidate key).

wing_name $\rightarrow$ {wing_id, type, floors}

Is wing_name a superkey? YES (it is a candidate key).

$\rightarrow$ Wing is in BCNF.

ROOMS (Weak Entity)

Candidate keys : {wing_id, room_no}

FDs:

{wing_id, room_no} $\rightarrow$ {capacity, availability} $\rightarrow$ {wing_id, room_no} is the superkey.

$\rightarrow$ Rooms is in BCNF.

STUDENT

Candidate keys : {student_id}, {email}, {contact}

FDs:

student_id $\rightarrow$ all attributes $\rightarrow$ student_id is a superkey.

email $\rightarrow$ student_id $\rightarrow$ email is a candidate key (superkey).

contact $\rightarrow$ student_id $\rightarrow$ contact is a candidate key (superkey).

$\rightarrow$ Student is in BCNF.

VISITOR

Candidate keys : {visitor_id}, {aadhar_no}

FDs:

visitor_id $\rightarrow$ all attributes $\rightarrow$ visitor_id is a superkey.

aadhar_no $\rightarrow$ visitor_id $\rightarrow$ aadhar_no is a candidate key (superkey).

$\rightarrow$ Visitor is in BCNF.

ADMIN

Candidate keys : {admin_id}, {email}

FDs:

admin_id $\rightarrow$ {name, email} $\rightarrow$ admin_id is a superkey.

email $\rightarrow$ admin_id $\rightarrow$ email is a candidate key (superkey).

→ Admin is in BCNF.

STAFF

Candidate keys : {staff_id}

FDs:

staff_id → {All Attributes}

staff_id is a superkey.

→ Staff is in BCNF.

WARDEN

Candidate keys : {staff_id}

FDs:

staff_id → assigned_wing → staff_id is a superkey.

→ Warden is in BCNF.

CLEANING_STAFF

Candidate keys : {staff_id}

FDs:

staff_id → working_area → staff_id is a superkey.

→ Cleaning_Staff is in BCNF.

PLUMBER

Candidate keys : {staff_id}

FDs : None (no non-key attributes).

→ Plumber is in BCNF.

CARPENTER

Candidate keys : {staff_id}

FDs : None (no non-key attributes).

→ Carpenter is in BCNF.

ELECTRICIAN

Candidate keys : {staff_id}

FDs : None (no non-key attributes).

→ Electrician is in BCNF.

ATTENDANCE

Candidate keys : {student_id, date}

FDs:

{student_id, date} → status → {student_id, date} is the superkey.

→ Attendance is in BCNF.

OVERNIGHT_LEAVE

Candidate keys : {leave_id}

FDs:

leave_id → {All Attributes} → leave_id is a superkey.
→ Overnight_leave is in BCNF.

COMPLAINT

Candidate keys : {comp_id}

FDs:

comp_id → {All Attributes} → comp_id is a superkey.
→ Complaint is in BCNF.

PARCEL

Candidate keys : {All Attributes}

FDs:

→ Parcel is in BCNF.

SUGGESTION

Candidate keys : {suggestion_id}

FDs:

suggestion_id → {description, date, Visitor_id} → suggestion_id is a superkey.
→ Suggestion is in BCNF.

PAYMENT

Candidate keys : {transaction_id}

FDs:

transaction_id → method, date → transaction_id is a superkey.
→ Payment is in BCNF.

STUDENT_PAYMENT

Candidate keys : {transaction_id}

FDs:

transaction_id → {student_id, Amount} → transaction_id is a superkey.
→ Student_Payment is in BCNF.

VISITOR_PAYMENT

Candidate keys : {transaction_id}

FDs:

transaction_id → {visitor_id, Amount} → transaction_id is a superkey.
→ Visitor_Payment is in BCNF.

PART 3 — MINIMAL FD SET

The minimal (canonical) FD set satisfies:

1. Each FD has a single attribute on the right-hand side.
2. No FD can be removed without changing the closure.
3. No attribute in any LHS is extraneous.

admin_id	→ name	
admin_id	→ email	
staff_id	→ name	
staff_id	→ contact_no	
staff_id	→ working_hours	
staff_id	→ working_days	
staff_id	→ salary	
staff_id	→ role	
staff_id	→ managed_by	
staff_id	→ assigned_wing	[Warden]
staff_id	→ working_area	[Cleaning_Staff]
wing_id	→ wing_name	
wing_id	→ type	
wing_id	→ floors	
wing_name	→ wing_id	
wing_name	→ type	
wing_name	→ floors	
room_no	→ capacity	
room_no	→ availability	
room_no	→ wing_id	
student_id	→ name	
student_id	→ dob	
student_id	→ contact	
student_id	→ email	
student_id	→ branch	
student_id	→ emergency_contact	
student_id	→ father_contact	
student_id	→ father_name	
student_id	→ year	
student_id	→ health_details	
student_id	→ status	
student_id	→ program	
student_id	→ resident_of	
student_id	→ hmc_flag	
student_id	→ gender	
{student_id, date}	→ status	[Attendance]
leave_id	→ request_date	
leave_id	→ from_date	

leave_id	→ to_date	
leave_id	→ return_timestamp	
leave_id	→ status	
leave_id	→ student_id	
leave_id	→ approved_by	
comp_id	→ type	
comp_id	→ status	
comp_id	→ description	
comp_id	→ reporting_date	
comp_id	→ assigned_by	
comp_id	→ student_id	
comp_id	→ solved_by	
parcel_id	→ arrival_date	
parcel_id	→ timestamp	
parcel_id	→ status	
parcel_id	→ received_date	
parcel_id	→ student_id	
parcel_id	→ handover_by	
visitor_id	→ name	
visitor_id	→ contact_no	
visitor_id	→ aadhar_no	
visitor_id	→ stay_duration	
visitor_id	→ check_in	
visitor_id	→ check_out	
visitor_id	→ request_status	
visitor_id	→ approved_by	
visitor_id	→ allocated_room	
aadhar_no	→ visitor_id	
suggestion_id	→ description	
suggestion_id	→ date	
suggestion_id	→ visitor_id	
transaction_id	→ method	[Payment]
transaction_id	→ date	[Payment]
transaction_id	→ student_id	[Student_Payment]
transaction_id	→ amount	[Student_Payment]
transaction_id	→ visitor_id	[Visitor_Payment]
transaction_id	→ amount	[Visitor_Payment]